



AULA 2

Redes neurais profundas: arquiteturas avançadas



Objetivos de aprendizagem da aula

Ao final desta aula, você irá:

- Entender para que servem as redes neurais recorrentes (RNNs) e como aplicar as *long short term memory networks* (LSTMs).
- Apresentar problemas que utilizam redes *autoencoders*.
- Discutir os principais conceitos das *generative adversarial networks* (GANs).
- Apresentar o funcionamento das principais aplicações de visão computacional e processamento de linguagem natural.

Que história é essa?

Você viu, na aula anterior, como funcionam as redes neurais artificiais e suas principais características. Entendeu como pequenas unidades simples (chamadas *perceptrons*) podem ser agrupadas para resolver problemas complexos. Também conheceu as redes neurais profundas e suas novas organizações em camadas para resolver problemas ainda mais robustos.

Nesta aula, você vai aprender outras arquiteturas de redes neurais profundas e entender como e quando devem ser aplicadas.

Redes neurais recorrentes

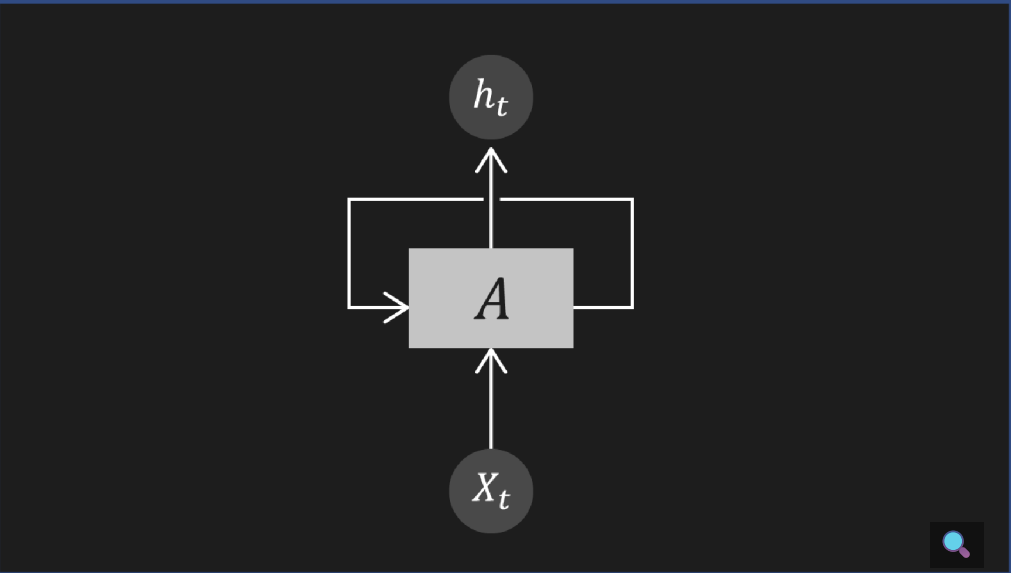


Interativo

Descrição do interativo

As saídas dependem apenas da amostra de entrada e do processamento feito pelos *perceptrons*. Elas são adequadas à maioria dos problemas em que a entrada é pontualmente processada pela rede, resultando em uma saída. Existem problemas em que o processamento do sinal deve considerar o histórico dos valores de entrada.

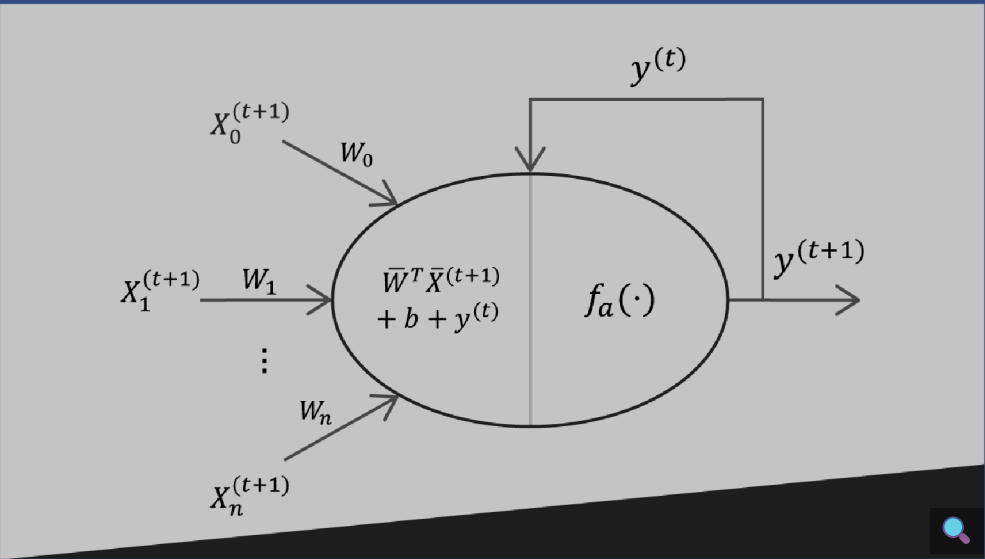
Veja as séries temporais: cada dado é ordenado em função do tempo e sofre influência de eventos anteriores. Exemplos: temperatura, áudio, vídeo, etc.



Nesses casos, o processamento do tempo t deve considerar a tendência ou, ao menos, a influência do passado recente em $t-1$. Foi justamente pensando nesses casos que se criaram as **redes neurais recorrentes (RNNs)**.

As RNNs constituem uma ampla classe de redes, cuja evolução do estado depende tanto da entrada corrente quanto do estado atual.

Nas RNNs, o neurônio não é mais uma unidade computacional de alimentação direta. A conexão de *feedback* o força a lembrar o passado e usá-lo para prever novos valores.



$$\begin{cases} y^{(t+1)} = f_a \left(\overline{w}^T \overline{x}^{(t+1)} + b + y^{(t)} \right) \text{ para } t > 0 \\ y^{(0)} = 0 \end{cases}$$

Nos *perceptrons* das RNNs, existe uma sinapse que recebe a previsão da rodada anterior e a adiciona à nova saída linear. O valor resultante é transformado pela função de ativação, para produzir a nova saída real. Convencionalmente, a primeira saída é nula, mas isso não é uma restrição.



Fique ligado

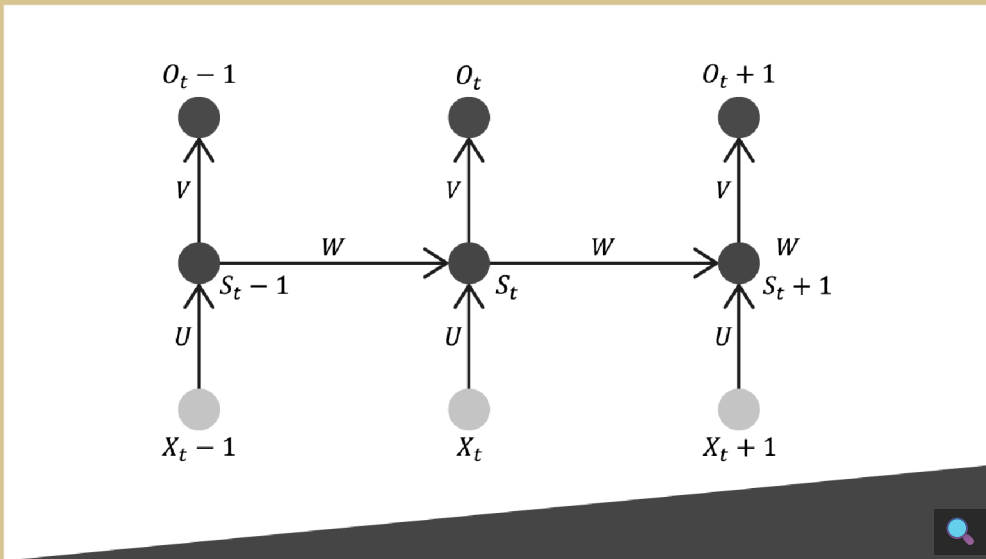
Por ser um sistema dinâmico, deve-se tomar cuidado com funções de ativação que saturam o sinal (como Sigmoid e Tangente Hiperbólica).

Essas saídas podem facilmente tornar o *perceptron* instável, pois, qualquer que seja, a saída pode “explodir”, movendo-se para $+\infty$ ou $-\infty$.

Desdobramento no tempo (*unfolding in time*)

Já que **os dados** que você aprenderá a processar **são séries temporais**, as **entradas da RNN são janelas de tempo** que serão relacionadas com a saída desejada. A entrada que recebe essas janelas se chama desdobramento no tempo e é uma sequência de valores, um em cada passo de tempo.

Você pode entender uma RNN que recebe uma sequência de n vetores de entrada como uma rede *feedforward* com n camadas, cujas matrizes de pesos são compartilhadas a cada passo de tempo.

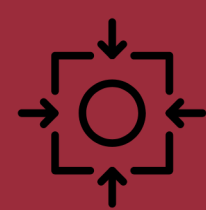


Backpropagation pelo tempo

O **algoritmo BPTT** é a extensão do *backpropagation*-padrão para redes com desdobramento. Ele visa **computar as saídas** da RNN e

determinar o valor da função de custo para cada rede.





Mantendo o foco

Assim como o *backpropagation*, ele calcula as correções dos pesos sinápticos (por meio dos gradientes) até a primeira camada da rede. Cada contribuição é baseada em uma experiência temporal precisa, composta por uma amostra local e um elemento de memória anterior.

O desdobramento no tempo permite o aprendizado de mapeamentos entre sequências de entrada e saída com diferentes tamanhos, clique nas flechas para explorar o assunto:

Interativo

Descrição do interativo

Rede LSTM

A rede LSTM (*long short term memory*) é análoga ao comportamento das memórias de curto e longo prazo cerebrais.



Sua principal ideia é manter uma memória de curto prazo que define o contexto e a memória de trabalho, enquanto a memória de longo prazo define o aprendizado do conhecimento adquirido.

Foi proposta em 1997, por Sepp Hochreiter e Jürgen Schmidhuber, e, apesar dos anos, continua sendo o estado da arte em redes recorrentes.

A LSTM é adequada quando há intervalos muito longos de tamanho desconhecido entre eventos importantes.



Fique ligado

Durante o treinamento, uma rede LSTM pode aprender quando deve manter ou descartar informações.

Em comparação, as redes recorrentes clássicas são focadas no aprendizado, mas não no esquecimento seletivo.

Dessa forma, a **LSTM** pode **otimizar a memória**, para lembrar o que é realmente importante, e remover todas as informações que não são necessárias, para prever novos valores.

Para realizar suas tarefas, a LSTM propõe explorar dois recursos importantes: estado e portões.

Estado	↓
É um conjunto separado de variáveis que armazenam os elementos necessários para construir dependências de longo e curto prazo. Inclui as informações do estado corrente e as variáveis responsáveis pelo gerenciamento cíclico e interno do erro fornecido pelo algoritmo de retropropagação.	
Portão	↓
Atua como um elemento que pode modular a quantidade de informação que flui por ele. Por exemplo, você pode supor que a equação $y = ax$ define um portão. A variável $a \in [0, 1]$ pode ser considerada o estado desse portão. Quando é igual a 0, o portão bloqueia a entrada x , e quando é igual a 1, permite que a entrada flua sem restrições.	

Qualquer valor intermediário reduz a quantidade de informação proporcionalmente, mas permite que a informação passe por esse canal.

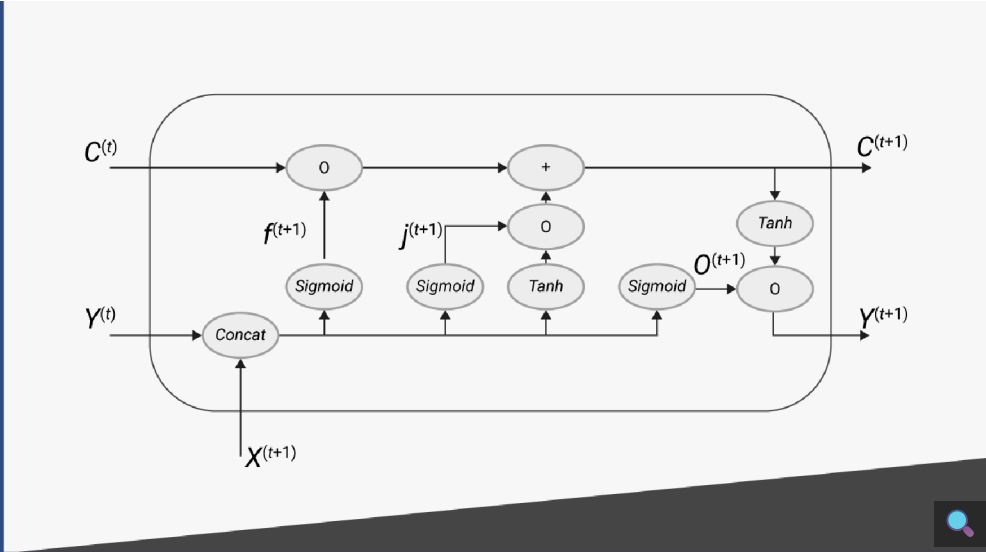


Fique ligado

Note que os portões são gerenciados por funções sigmoides, enquanto as ativações são baseadas em tangentes hiperbólicas, cuja simetria garante melhores desempenhos.

Bloco LSTM

O bloco LSTM é a **estrutura que implementa o controle** das memórias de longo e curto prazo e seus estados internos.



Quando os portões estão “fechados”, os gradientes podem fluir pela unidade de memória sem alteração, por tempo indefinido.



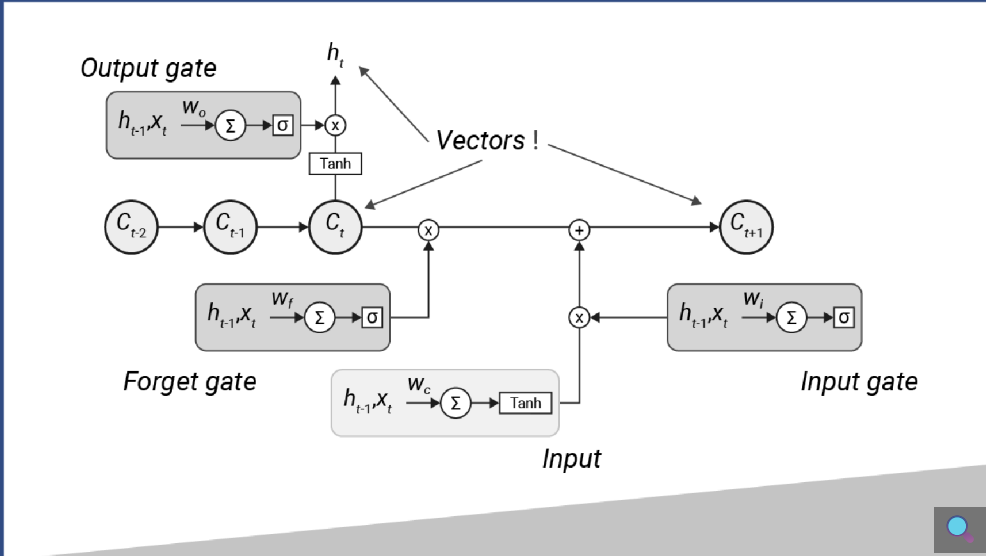
Fique ligado

Embora os portões nunca isolem a unidade de memória completamente, as LSTMs abordam o problema da dissipação dos gradientes em, pelo menos, algumas situações.

Há três portões em um bloco LSTM:

- Forget gate
- Memory gate (input gate)
- Output gate

Cada portão produz valores computados por uma RNA *feedforward* (interna à célula) de uma única camada.



O **estado da memória** é responsável pelas **dependências e pela saída real**. O estado atual depende do valor anterior, da entrada atual e da saída anterior. O novo estado, $c^{(t+1)}$, é calculado pela soma do valor gerado pelo portão de esquecimento, g_1 , e pelo portão de memória, g_2 .

$$C^{(t+1)} = g_1 \left(C^{(t)} \right) + g_2 \left(\bar{x}^{(t+1)}, \bar{y}^{(t)} \right)$$



Portão de esquecimento

O portão de esquecimento é responsável pela **persistência dos elementos** de memória existentes ou pela sua eliminação.

$$\bar{f}^{(t+1)} = \sigma \left(W_f \cdot \left(\frac{\bar{y}^{(t)}}{\bar{x}^{(t+1)}} \right) + \bar{b}_f \right)$$

Veja que o vetor de pesos sinápticos, W_f , é responsável por processar o sinal atual e o sinal recebido da saída do turno anterior (recorrência).

A função sigma define a função de ativação sigmoide.

Para determinar o valor que passa pelo portão de esquecimento, $g_1 \left(C^{(t)} \right)$, usa-se o produto de Hadamard:

$$g_1 \left(C^{(t)} \right) = \bar{f}^{(t+1)} \circ C^{(t)}$$

Perceba que o efeito de $g_1 \left(C^{(t)} \right)$ é filtrar o valor de $C^{(t)}$ em relação ao grau de validade, que é proporcional ao valor de $f^{(f+1)}$.

Se o portão de esquecimento emitir um valor próximo a 1, o elemento correspondente ainda é considerado válido.

Em caso de valores mais baixos, estes determinam uma espécie de obsolescência até a célula remover completamente um elemento, quando o valor do portão de esquecimento for 0.

Portão de memória

O portão de memória é **responsável por considerar** a quantidade da amostra de entrada que deve ser usada para atualizar o estado. Da mesma forma que o portão de esquecimento, a função $i^{(t+1)}$ processa o sinal de entrada atual e a recorrência da saída do turno anterior.

Esse valor é combinado à proposta de um novo contexto C, que é calculado e processado por uma função de ativação tangente hiperbólica.

$$\begin{aligned} \bar{i}^{(t+1)} &= \sigma \left(W_i \cdot \left(\frac{\bar{y}^{(t)}}{\bar{x}^{(t+1)}} \right) + \bar{b}_i \right) \\ \hat{C}^{(t+1)} &= \tanh \left(W_c \cdot \left(\frac{\bar{y}^{(t)}}{\bar{x}^{(t+1)}} \right) + \bar{b}_c \right) \end{aligned}$$

Usando a porta de entrada e a contribuição de estado, é possível determinar a função $g_2 \left(x^{(t+1)}, y^{(t)} \right)$:

$$g_2 \left(\bar{x}^{(t+1)}, \bar{y}^{(t)} \right) = \bar{i}^{(t+1)} \circ C^{(t+1)}$$

Assim, o novo contexto é calculado pela soma do resultado do portão de esquecimento com o resultado do portão de memória:



$$C^{(t+1)} = \left(\overline{f}^{(t+1)} \circ C^{(t)}\right) + \left(\overline{i}^{(t+1)} \circ \hat{C}^{(t+1)}\right)$$

Perceba que a lógica interna de uma célula LSTM é um equilíbrio dinâmico entre a experiência anterior e sua reavaliação de acordo com a nova experiência (modulada pelo portão de esquecimento), adicionados ao efeito semântico da entrada de corrente (modulada pela porta de entrada $i^{(t+1)}$).

Portão de saída

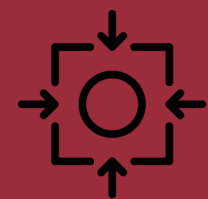
Por fim, a porta de saída **controla as informações** que devem transitar do estado para a unidade de saída. O sinal de saída potencial é calculado exatamente como nas portas anteriores.

$$\overline{o}^{(t+1)} = \sigma \left(W_o \cdot \left(\frac{\overline{y}^{(t)}}{\overline{x}^{(t+1)}} \right) + \overline{b}_o \right)$$

O portão de saída é controlado pelo contexto atual, fornecido pela combinação dos dois portões anteriores.

$$\overline{Y}^{(t+1)} = \overline{o}^{(t+1)} \circ \tanh \left(c^{(t+1)} \right)$$

Perceba que todos os portões são alimentados pelos vetores da entrada atual e da saída anterior.



Mantendo o foco

O portão de esquecimento expressa a probabilidade de a última sequência ser mais importante que o estado atual. Já o portão de saída expressa a probabilidade de a sequência atual ser capaz de deixar o estado atual fluir. Apenas o portão de entrada $i^{(t+1)}$ tem a responsabilidade de conceder a ela o direito de influenciar o novo estado.

Você pode entender que os blocos LSTM têm operações para leitura, gravação e reiniciação, responsáveis por manipular a memória.

Apesar de obterem excelentes respostas, preste atenção a algumas questões.

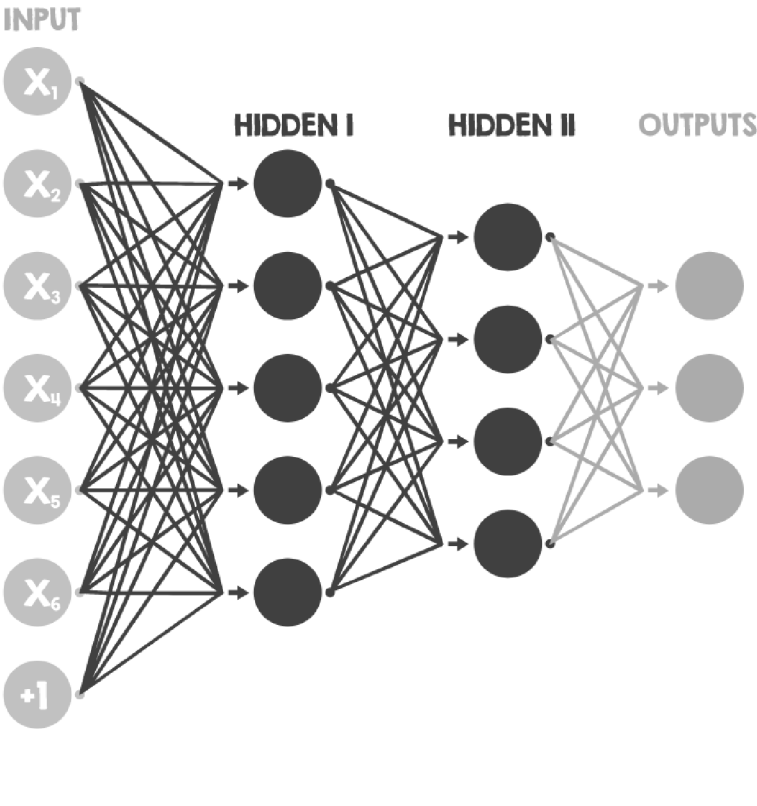


Fique ligado

Se a porta de saída permanecer fechada, a saída tende a zero, e isso influencia as portas de esquecimento e entrada.

Como o esquecimento e a entrada controlam o novo estado, podem limitar a quantidade de informações recebidas e suas conseqüentes correções, levando a um desempenho ruim.

Autoencoders



Os *autoencoders* são arquiteturas que oferecem uma **abordagem diferente para problemas clássicos** de redução de dimensionalidade ou aprendizado de dicionários.

Por serem adaptativos, não sofrem as limitações que afetam modelos estatísticos clássicos. Você pode utilizá-los para explorar camadas neurais específicas e extrair informações dos dados, definindo critérios específicos (por exemplo, características de rostos de pessoas no *dataset*).



Fique ligado

A rede cria representações internas que podem ser mais robustas a diferentes tipos de distorções e muito mais eficientes, por permitirem uma maior quantidade de informações a serem processadas.

Sabe-se que, em um conjunto de dados, cada coluna representa uma dimensão em um espaço vetorial complexo. Muitas vezes, as amostras são representadas em uma alta dimensão, mas também podem ser expressas em uma dimensão menor, sem perda de informação.

Esse conceito é importante, pois a complexidade de um modelo é proporcional à dimensionalidade dos dados de entrada. Existem muitas técnicas estatísticas para tentar reduzir o número real de

componentes válidos. Na próxima aula, você vai entender mais sobre elas.

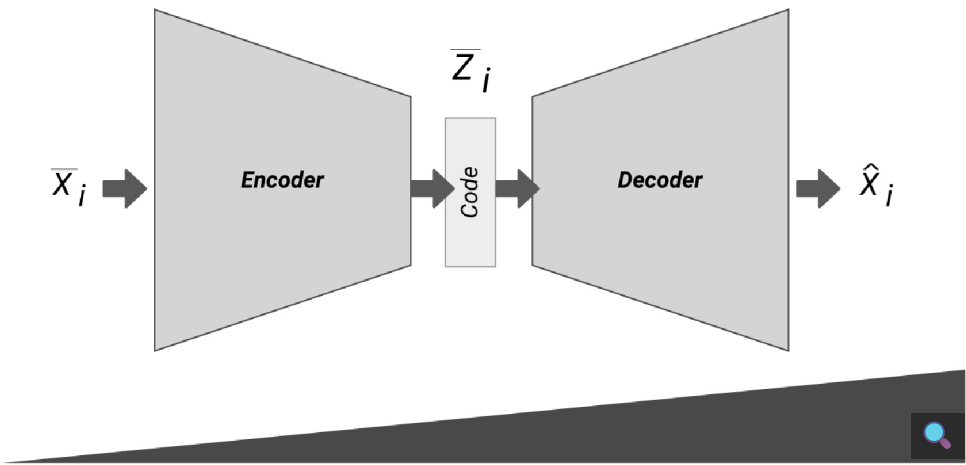


Um *autoencoder* genérico é um modelo dividido em dois componentes separados: *encoder* e *decoder*. Eles não funcionam completamente autônomos e são treinados juntos.

Interativo

Descrição do interativo

Observe a imagem a seguir:



Pode-se definir o codificador como uma função parametrizada, em que a saída z_i é um “código” vetorial de dimensionalidade bem menor que as entradas:

$$\bar{z}_i = e(\bar{x}_i; \bar{\theta}_e) \text{ em que } \bar{x}_i \in X$$

...em que X é o conjunto de amostras $\bar{\theta}_e$, o conjunto de pesos aprendidos pelo modelo de codificação durante o treinamento.

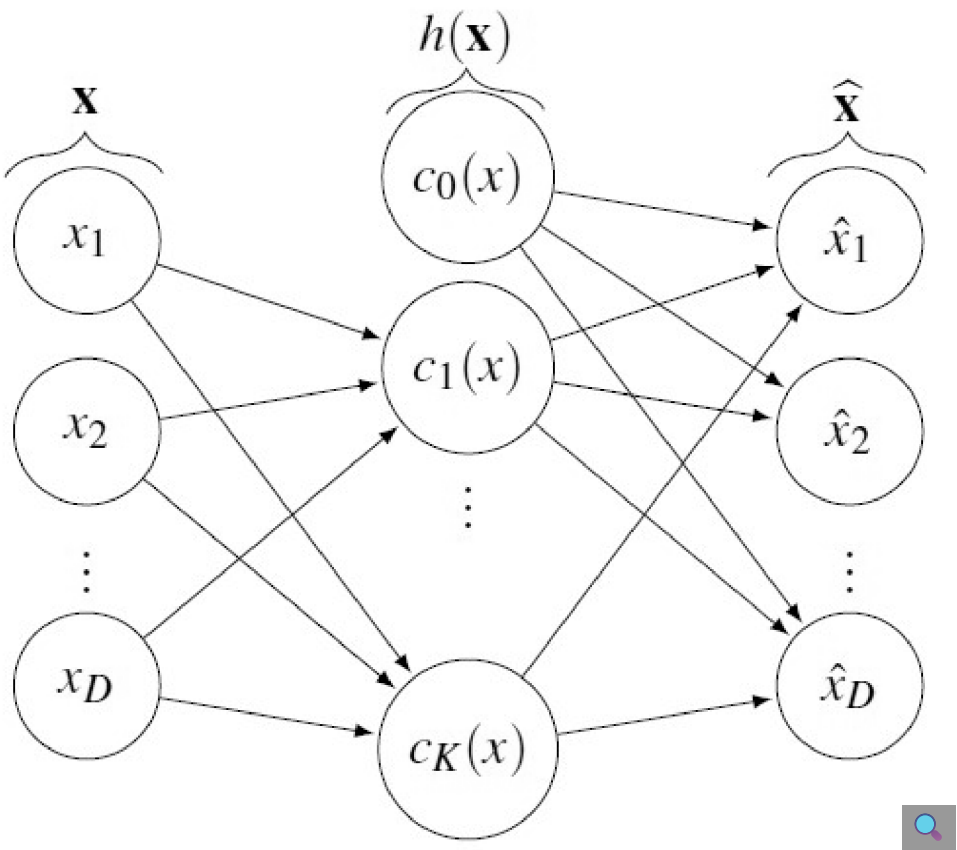
Da mesma forma, pode-se entender o decodificador como outra função, em que a saída x_i é um vetor reconstruído, de dimensionalidade igual ao vetor original:

$$\hat{x}_i = d(\bar{z}_i; \bar{\theta}_d)$$

...em que $\bar{Z}_i$ é o conjunto de variáveis codificadas e $\bar{\theta}_d$, o conjunto de pesos aprendidos pelo modelo de decodificação durante o treinamento.



Durante o aprendizado, uma rede *autoencoder* é vista como um par de redes acopladas: uma codificadora e a outra decodificadora.



Pode-se entender que a rede codificadora x realiza a “compressão” dos dados de entrada e a rede decodificadora, a “descompressão” $\hat{x}$ para a sua forma original.

A representação codificada, muitas vezes, é chamada de variáveis latentes, pois cada dimensão pode representar uma característica da amostra original.

Denoising autoencoders

Devido às suas propriedades, os *autoencoders* podem ser **usados para auxiliar representações incompletas** de um conjunto de dados.

Como podem aprender a representação exata de uma amostra, a fim de reconstruí-la a partir de um código de baixa dimensão, também ajudam a eliminar o ruído das amostras de entrada.

Nesse caso, entende-se que o codificador deve trabalhar com amostras ruidosas, que são acrescidas de uma variável de erro (ruído gaussiano) $\bar{n}_i$.

$$\bar{z}_i = e\left(\bar{x}_i + \bar{n}_i(t); \bar{\theta}_e\right) \text{ em que } \bar{x}_i \in X \text{ e } \bar{n}_i(t) \sim N(0, \sigma^2)$$

Dessa forma, o codificador se torna robusto e aprende a codificar as amostras, mesmo quando a informação foi incompleta ou ruidosa. Já a função de custo do decodificador permanece a mesma, pois deve reconstruir a informação original.



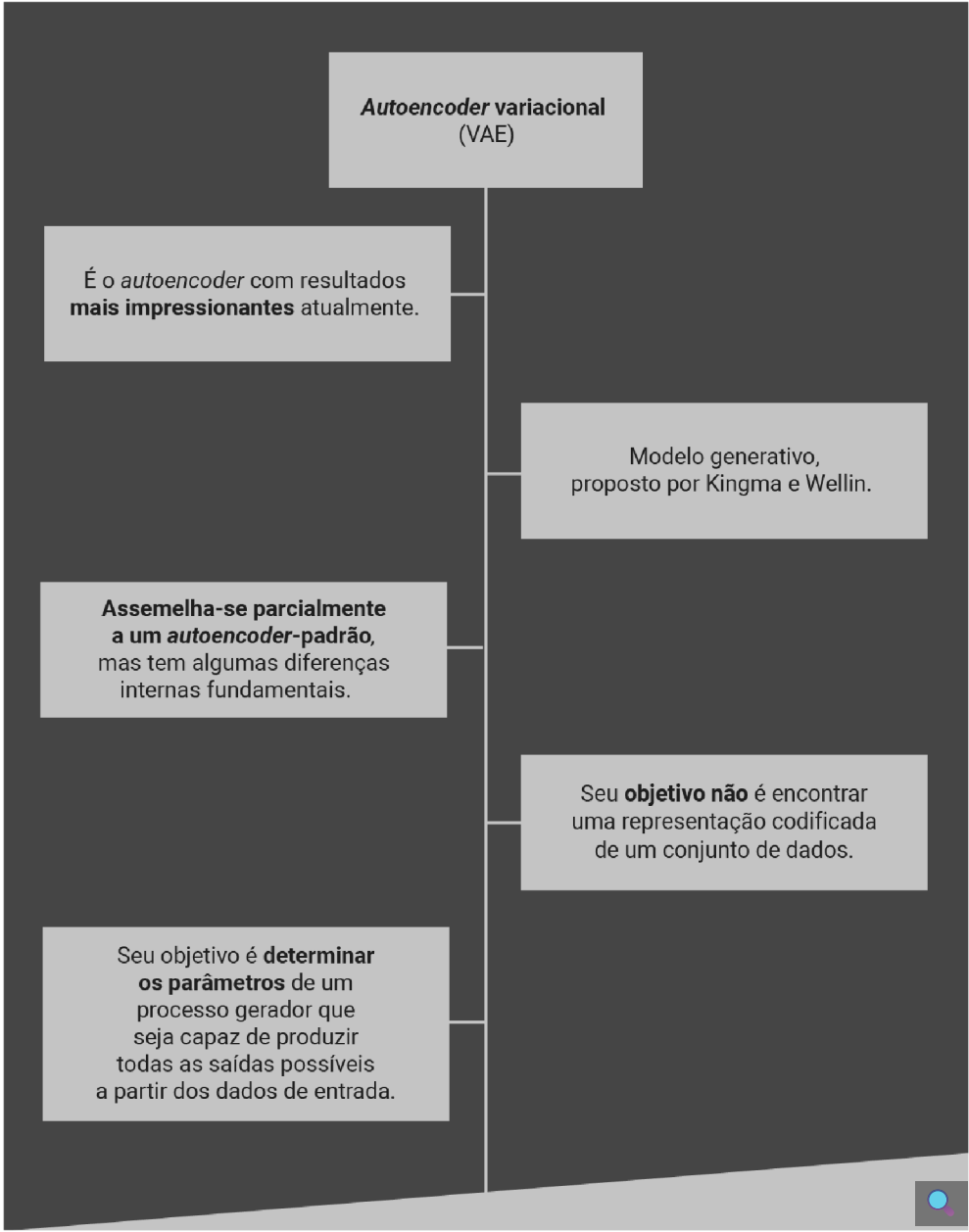
Fique ligado

Assim, você faz o treinamento da codificação de uma informação perfeita, adicionando ruído artificialmente à entrada do codificador, e sua reconstrução para a forma original, pelo decodificador.

É necessário um número suficientemente grande de iterações para permitir que o *autoencoder* aprenda a reconstruir a imagem original quando alguns fragmentos estiverem ausentes ou corrompidos. Pode-se usar uma camada de *dropout* como alternativa de entrada do codificador, para simular o preenchimento ruidoso da amostra:

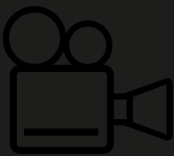
$$\bar{z}_i = e\left(\bar{x}_i \circ \bar{n}_i; \bar{\theta}_e\right) \text{ em que } \bar{x}_i \in X \text{ e } \bar{n}_i(x, y) \sim B(0, 1)$$

Variational autoencoders



O *autoencoder* com resultados mais impressionantes, atualmente, é o *autoencoder* variacional (VAE). Ele é um modelo generativo, proposto por Kingma e Wellin, que se assemelha parcialmente a um *autoencoder*-padrão, mas tem algumas diferenças internas fundamentais.

Seu objetivo não é encontrar uma representação codificada de um conjunto de dados, mas determinar os parâmetros de um processo gerador que seja capaz de produzir todas as saídas possíveis a partir dos dados de entrada.



Exemplos de *autoencoders*

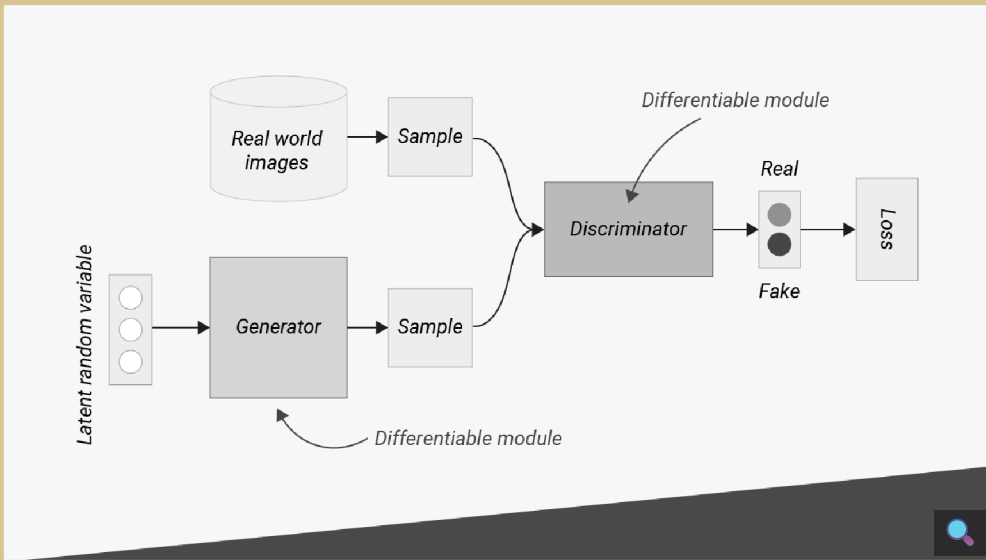
Neste vídeo você terá a oportunidade de retomar sobre os *autoencoders*, verá exemplo desta aplicação, e por fim, verá também exemplos de aplicação de *denoising autoencoders* e *variational autoencoders*.



GANs (redes adversárias generativas)

A GAN é uma arquitetura composta de duas redes neurais que **competem entre si**.

A primeira é uma rede geradora, que deve ser capaz de gerar/criar informações o mais fiéis possível às características das amostras usadas durante o treinamento. A outra rede deve aprender a criticar suas entradas e decidir se os dados têm características que os tornem verdadeiros ou se seriam falsos e gerados artificialmente.



A ideia por trás dessa organização é que uma rede motiva o treinamento da outra e ambas avançam no treinamento dos dados. Por exemplo, uma GAN treinada em fotografias pode gerar novas imagens que parecem autênticas (ao menos superficialmente) para observadores humanos, apresentando muitas características realistas.

10/11/2025, 21:39

Redes neurais profundas: arquiteturas avançadas

1

Gerador: gera amostras falsas, tenta enganar o discriminador.

2

Discriminador tenta distinguir entre amostras reais e falsas.

3

Treinam-se uns contra os outros.

4

Após repetições, obtêm-se um gerador e um discriminador melhores.

Processo de treinamento

O treinamento contraditório (*adversarial training*) visa **aprender a distinguir** entre amostras verdadeiras e falsas, fazendo com que o componente que gera amostras se aproxime cada vez mais dos exemplos de treinamento.

Fique ligado

A ideia de treinamento contraditório é uma abordagem parcialmente inspirada na teoria dos jogos (algoritmo *minmax* para jogos competitivos).

Na prática, a ideia é bem simples: primeiro, o gerador deve criar uma amostra x .

$$\hat{x}_i = G(\bar{z}_i; \bar{\theta}_g) \text{ em que } \bar{z}_i \sim U(-1, 1)$$

...em que Z_i é um vetor de variáveis latentes criado aleatoriamente e θ , o vetor de pesos aprendidos durante o treinamento.

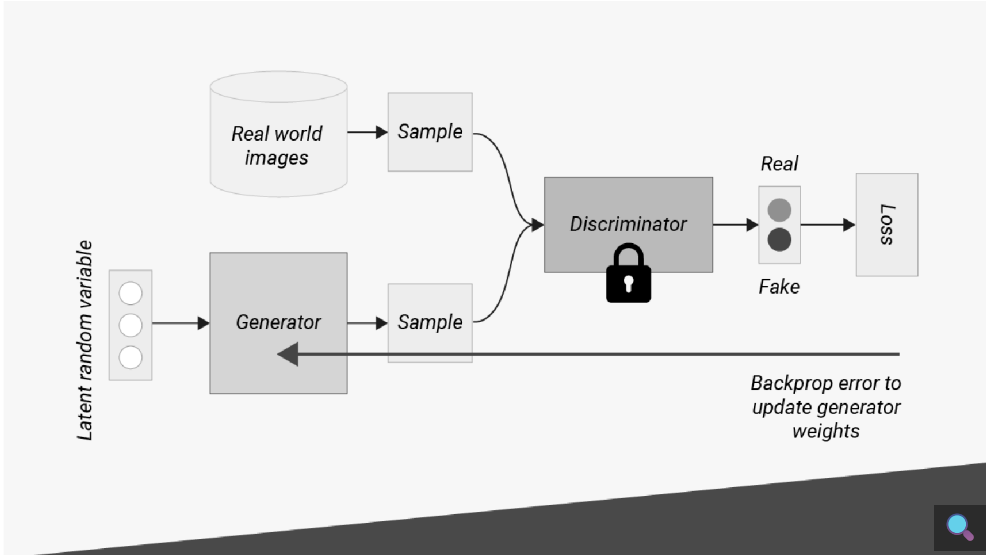
O discriminador (ou crítico) deve calcular a probabilidade de os dados do vetor x serem verdadeiros.

$$p_i = D(\bar{x}_i; \bar{\theta}_d) \text{ where } p_i \in [0, 1]$$

Se x realmente pertencer ao conjunto de dados treinados, o discriminador produz um valor próximo a 1. Caso seja muito diferente das outras amostras verdadeiras, $D(x; \theta_d)$ produzirá uma probabilidade muito baixa.

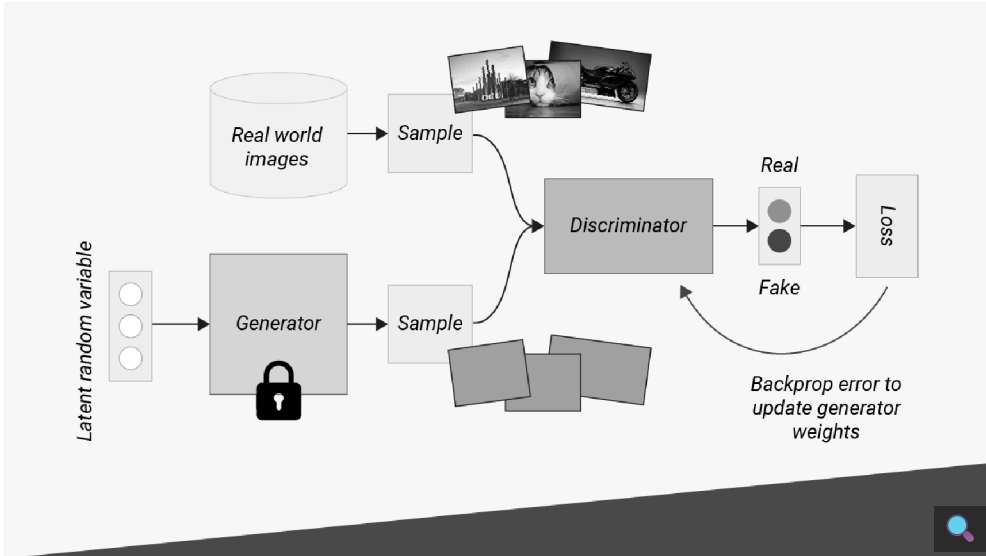
https://pucrio.grupoa.education/plataforma/course/2565697

14/16



Nessa etapa, a ideia do jogo de competição é criar um ambiente para treinar o gerador para enganar o discriminador, produzindo amostras que potencialmente podem ser extraídas do conjunto de dados reais.

Em seguida, quando o gerador é bom o suficiente para enganar o discriminador, é sua vez de treinar para melhorar sua avaliação.



Após repetições, obtêm-se um gerador e um discriminador melhores. A primeira arquitetura da família GAN proposta foi a Deep Convolutional Generative Adversarial Network, fortemente influenciada pelas arquiteturas que você viu anteriormente.

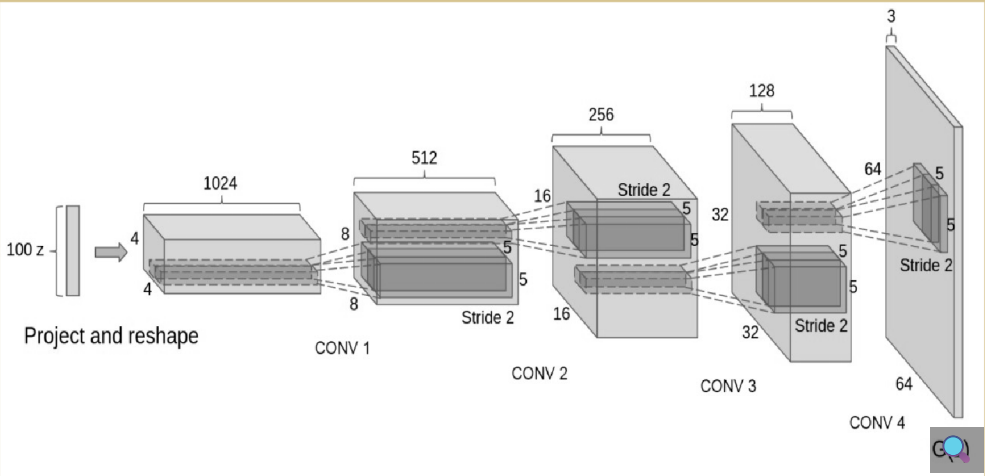
Deep Convolutional GANs (DCGANs)

As DCGANs foram propostas no artigo “*Unsupervised representation learning with deep convolutional generative adversarial networks*” (RADFORD; METZ; CHINTALA, 2016), e seu gerador tem quatro convoluções de transposição, com tamanhos de kernel iguais a (4, 4) e stride (2, 2).

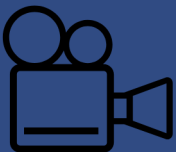


Fique ligado

A entrada é um único *pixel* multicanal ($1 \times 1 \times code_length$), expandido por convoluções subsequentes.



Os autores ainda sugeriram o uso de um conjunto de dados de valor simétrico e, por isso, normalizam-se os valores entre -1 e 1 em lote após cada camada convolucional. Também se utiliza a função de ativação Leaky ReLU (com *slope* 0,2).



Redes adversárias generativas

Neste vídeo você irá conhecer um pouco mais sobre as redes adversárias generativas e seu processo de treinamento.



Referências

Clique aqui para acessar as referências e créditos desta aula.



BONACCORSO, G. **Machine learning algorithms**. 2. ed. Birmingham: Packt Publishing, 2018.

BONACCORSO, G. **Mastering machine learning algorithms**. Birmingham: Packt Publishing, 2018.

GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. **Deep learning**. Cambridge, MA: MIT Press, 2016.

OLAH, C. Deep learning, nlp, and representations. **GitHub blog**, 2015. Disponível em: <http://colah.github.io/posts/2015-08-Understanding-LSTMs/>. Acesso em: 23 mar. 2023.

RADFORD, A.; METZ, L.; CHINTALA, S. Unsupervised representation learning with deep convolutional generative adversarial networks. 2015. Disponível em: <https://doi.org/10.48550/arXiv.1511.06434>. Acesso em: 24 mar. 2023.

Banco de imagens Pexels, Pixabay, Envato, Freepik e Shutterstock.